Simulating 100,000 VUs & Defending Against HTTP Floods

A practical, battle-tested engineering blueprint for Layer 7 load-testing with k6 and multi-tier edge & origin hardening with Cloudflare and Nginx.

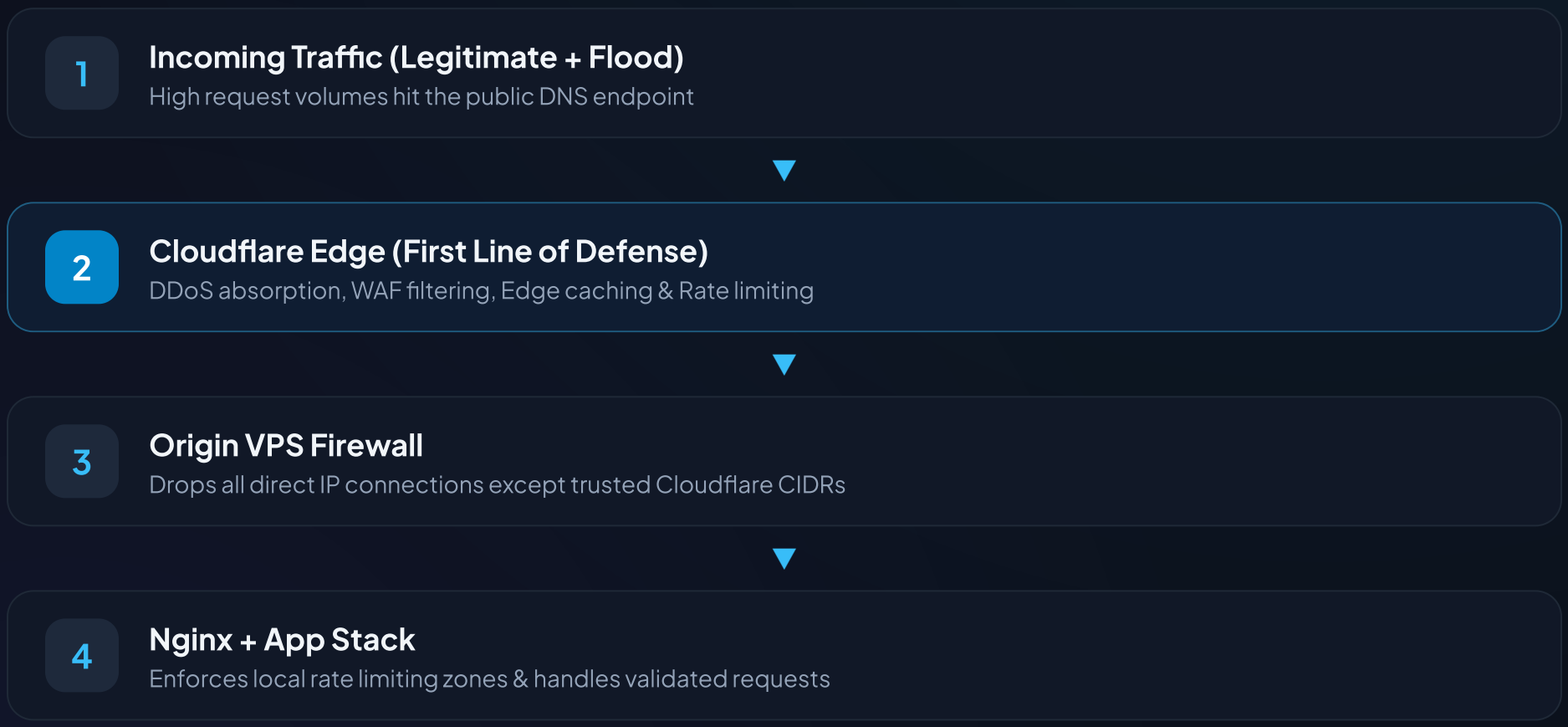
What You Will Learn

- Generating ramping 100k VU traffic with k6
- Cloudflare WAF & Managed Challenge configuration
- Preventing the fatal "Direct Origin Bypass"
- Nginx rate limiting & application-layer protection

Aditya Jain
Security & Systems Architecture
adityasec32.systems
in/ajainx1

Understanding Layer 7 HTTP Floods

Unlike L3/L4 volumetric attacks that saturate bandwidth, an HTTP flood mimics legitimate requests to exhaust application workers and databases.



Step 1: k6 Baseline Load Test

Never build defenses blind. Establish your baseline breaking point by ramping Virtual Users (VUs) before applying mitigation.

📈 Ramp Profile

1,000 ➔ 5,000 ➔ 10,000 ➔ 25,000 ➔ 50,000 ➔ 100,000 VUs

Holds peak for 10 minutes to verify connection pooling and worker latency.

```
import http from 'k6/http'; export default function () { const response = http.get('https://your-domain.com/', { tags: { name: 'Homepage' }, timeout: '30s', }); }
```

🔍 Live Server Monitoring Commands

```
htop | free -h | ss -s | tail -f /var/log/nginx/error.log
```

Step 2: Cloudflare WAF Rules

Filter bad actors and automated scrapers at the edge before they touch your server infrastructure.

🛡️ Rule 1: Geographic / ASN Managed Challenge

`(ip.geoip.country ne "IN")` → **Managed Challenge**

Presents interactive challenges for traffic outside your target audience without blocking legitimate international users.

🚫 Rule 2: Block High-Frequency Scanners

`(http.request.uri.path contains "/xmlrpc.php")` → **Block**

Instantly drops legacy CMS brute-force vectors and amplification exploits.

⚡ Rule 3: Drop Deprecated HTTP Versions

`(http.request.version eq "HTTP/1.0")` → **Block**

Stops automated flood scripts that fail to implement modern HTTP/2 or HTTP/3 multiplexing.

Step 3: Prevent Origin Bypass

The most common architecture failure: Leaving the VPS origin IP directly accessible to the internet.

⚠️ The Attack Vector

Attackers use Shodan, Censys, or historical DNS logs to discover your real VPS IP. If port 80/443 accepts traffic from `0.0.0.0/0`, they bypass all Cloudflare WAF and DDoS shields entirely.

🔒 The Solution: Strict Origin Firewall

- Configure UFW or Cloud Security Groups to **allow HTTP/HTTPS traffic ONLY from Cloudflare published IP ranges**.
- Drop all other direct requests to ports 80 & 443.
- Keep administrative ports (SSH, Control Panels) restricted to your personal static IP or dedicated VPN tunnel.

Step 4: Nginx Rate Limiting

Even behind Cloudflare, throttle bursts at the web server layer to protect database and PHP/Node worker pools.

```
# Define a 10MB memory zone (approx. 160,000 IPs) limit_req_zone $binary_remote_addr
zone=site_limit:10m rate=10r/s; server { location / { # Allow burst up to 20 requests without
delay limit_req zone=site_limit burst=20 nodelay; } }
```

💡 Crucial: Real-IP Restoration

Ensure Nginx has `ngx_http_realip_module` enabled and uses `CF-Connecting-IP`. Otherwise, Nginx will rate-limit Cloudflare's edge proxy IPs and block all legitimate visitors!

The 5 Golden Rules of L7 Defense

✅ **1. Measure First:** Run authorized k6 ramp tests to discover baseline limits and latency breaking points.

✅ **2. Edge First:** Absorb volume at Cloudflare; aggressively edge-cache static assets.

✅ **3. Block Origin Bypass:** Lock your VPS firewall strictly to Cloudflare IPs. Never expose ports 80/443 to the world.

✅ **4. Defense in Depth:** Pair Cloudflare WAF with Nginx `limit_req` and database query safeguards.

✅ **5. Retest & Verify:** Repeat the k6 stress test post-deployment to validate stability under peak traffic.

Aditya Jain
Follow for Systems Architecture & Security

🌐 adityasec32.systems
💼 linkedin.com/in/ajainx1